

Rivage: A Polyglot Distributed Compute Engine for Numerical and Batch Processing

A Report submitted
in partial fulfillment of the requirement
for the award of a Degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Your Name

(Your Roll Number)

Under the guidance of

Guide Name

**Department of Computer Science and Engineering
National Institute of Technology, Srinagar,
Kashmir 190006, INDIA**

May 2026

CHAPTER 0

CERTIFICATE

[To be written: Certificate of original work signed by the supervisor as per the NIT Srinagar format.]

CHAPTER 0

STUDENT DECLARATION

[To be written: Declaration that the work is original and free from plagiarism.]

CHAPTER 0

ACKNOWLEDGEMENTS

[To be written: Express gratitude to supervisor, department, and institution.]

CHAPTER 0

ABSTRACT

[To be written: A formal summary of the entire system, highlighting the zero-dependency Go architecture, polyglot execution, and fault tolerance.]

CHAPTER 0

TABLE OF CONTENTS

CERTIFICATE	III
STUDENT DECLARATION	IV
ACKNOWLEDGEMENTS	V
ABSTRACT	VI
LIST OF FIGURES	IX
LIST OF TABLES	X
ABBREVIATIONS / NOTATIONS / NOMENCLATURE	XI
1 INTRODUCTION	1
1.1 Overview	1
2 Literature Review	3
2.1 Evolution of Distributed Computation Paradigms	3
2.2 Middleware Tax and Polyglot Limitations	3
2.3 Fault Tolerance and State Recovery	3
2.4 Resource Routing and Communication Layers	4
2.5 Theoretical Limits of Distributed Computing	5
3 PROPOSED METHODOLOGY	6
3.1 Problem Statement	6
3.2 Problem Description and System Requirements	6
3.3 Background and Theoretical Foundations	8
3.4 System Architecture & Design	17
3.5 The Coordinator (Control Plane)	21
3.6 The Scheduler (Resource Allocation and Load Balancing)	29
3.7 The Worker Daemon (Compute Plane)	38
3.8 Fault Tolerance and System Resilience	43
3.9 The Programming Model and Programmable Shuffles	47

4	RESULTS AND DISCUSSION	51
5	CONCLUSION AND FUTURE WORKS	52
	REFERENCES	53
A.	APPENDIX A: Output	54

CHAPTER 0

LIST OF FIGURES

Figure 1	Theoretical data flow and functional boundaries of the MapReduce paradigm	11
Figure 2	Execution trace of a distributed Word Count workload. Network shuffle intersections are rendered cleanly using crossing halos.	14
Figure 3	Pseudocode representation of topological graph resolution via Kahn's Algorithm	16
Figure 4	Vertical topology of the Rivage Engine. Both the orchestration layer and the worker subprocess pipelines flow perfectly top-to-bottom.	18
Figure 5	The internal task state machine managed by the Coordinator.	26
Figure 6	Pseudocode representation of the asynchronous batched state evaluation loop (<code>watchJob</code>)	28
Figure 7	Execution flow and decision matrix of the Least-Loaded routing algorithm.	35
Figure 8	Part 1: The $O(N)$ filtering matrix isolating hardware constraints and maximizing data locality.	36
Figure 9	Part 2: Applying the least-loaded heuristic and physical CPU tie-breakers to select the final node.	37
Figure 10	The Worker Daemon's bounded-memory I/O pipeline. Data is continuously flushed to local disk to preserve RAM during polyglot execution.	40
Figure 11	Pseudocode representing the bounded-memory POSIX execution pipeline within the Worker Daemon.	42
Figure 12	Pseudocode representing the deterministic WAL recovery algorithm executed during master node reboot.	46

CHAPTER 0

LIST OF TABLES

CHAPTER 0

ABBREVIATIONS / NOTATIONS / NOMENCLATURE

[To be written: List of abbreviations like DAG, RPC, WAL, gRPC, OOM, etc.]

CHAPTER 1

INTRODUCTION

1.1 Overview

With growing technology, the amount of data grows, and with that grows the amount of compute we need to process the humongous data. CPUs and GPUs have a limit on how much they can process, and most normal computers/devices have a limit on how many processing units they can hold. Thus, processing so much data in a congested computer is tough. This breeds the need for distributed compute, i.e., using multiple scattered devices to do the computations.

Obviously, the tech industry has had these problems for a long time, and they have already figured out industry-grade solutions for them. These include systems like Apache Hadoop, which in general are very monolithic, complicated, hard to set up, and heavily associated to specific ecosystems like the Java Virtual Machine (JVM). This rigid architecture leads to steep learning curves, high memory footprints, and difficulty in rolling out lightweight scripts or using other programming languages without relying on inefficient wrappers. For many general-purpose distributed computing tasks, deploying such a heavy architecture is unnecessary and cumbersome.

To solve these challenges, this project introduces a distributed computing framework that is a lightweight, high-throughput distributed MapReduce orchestration engine. Rivage is built from the ground up using Go (Golang), leveraging the language's native concurrency model and high-performance networking capabilities. Our focus is on using modern technologies like gRPC, stackful coroutines, Protocol Buffers, etc.

This project uses a polyglot execution environment using scripts written in any language (generally simple scripting languages) to write the executable code while treating worker nodes as agnostic execution vessels. We use a mix of JSON and binary data along with a centralized data store to save and update the data. This design allows users to write the

map and reduce functions in the language of their choice while keeping the orchestration logic in Golang itself.

CHAPTER 2

Literature Review

2.1 Evolution of Distributed Computation Paradigms

The domain of distributed computing was transformed by the introduction of the MapReduce programming model by Dean and Ghemawat [1]. This led to the pioneering of data parallelization, fault tolerance, and data locality, allowing developers to orchestrate work in a value-oriented way without needing to think extensively about the underlying network topology. However, the MapReduce model carried a significant disk I/O bottleneck; due to the orchestration requirements, the output of one phase had to be written to disk before being used for the reduce phase, which introduced substantial unwanted latency.

To solve these problems, Zaharia et al. introduced Apache Spark and the concept of *Resilient Distributed Datasets (RDDs)* [2]. This led to a fundamental shift toward in-memory data sharing and computing, bypassing disk I/O entirely.

2.2 Middleware Tax and Polyglot Limitations

Enterprise-grade solutions like Apache Spark and Hadoop managed to solve the problems of distributed computing using the JVM and heavy dependency systems that are rewarding for enterprise-level workloads. Although perfect for large industries, this introduces what is known as a ‘middleware tax’ for most mid-to-low-level workloads. Added to this are the strict limitations on the languages available for writing tasks, making the process too tedious for most minor distributed workloads.

2.3 Fault Tolerance and State Recovery

Handling hardware volatility is one of the primary challenges in distributed computing. There are many levels at which hardware may fail, including networks and compute nodes; however, we are predominantly concerned with handling failures within the

compute systems. These can be of various types, but they can be broadly categorized based on the statefulness of the failed component:

- **Stateless Compute (Worker Nodes):** In a decoupled setup, worker nodes act as interchangeable compute engines. Handling a worker crash is straightforward because these machines do not store any permanent cluster state. By making tasks idempotent, the orchestrator does not waste time trying to recover the specific crashed hardware. It simply revokes the interrupted task and reassigns it to a healthy node in the cluster, a fault-tolerance pattern originally popularized by MapReduce [1].
- **Stateful Compute (Master Node/Control Plane):** Recovering the central orchestrator is fundamentally harder. A master node cannot be restarted from a blank slate since it actively maintains the DAG topology and the execution progress of the entire cluster. To survive a master crash, the system must record every state change to a highly durable log before actually dispatching tasks. This technique, Write-Ahead Logging (WAL), was formally established by Mohan et al. in the ARIES algorithm [3] and allows the master to deterministically reconstruct its exact operational state after a power loss.

2.4 Resource Routing and Communication Layers

The Remote Procedure Call (RPC) was originally introduced by Birrell and Nelson to help distributed programs communicate smoothly across networks [4]. While legacy cluster managers have relied on internal RPC implementations for years, modern frameworks like gRPC take a more streamlined approach by building on HTTP/2 and Protocol Buffers [5]. Instead of moving monolithic payloads across the network, gRPC relies on bounded-memory binary streaming, which makes concurrent telemetry and task dispatching far more efficient. Because this binary contract is strict and highly optimized, it requires almost zero infrastructure overhead. As a result, lightweight orchestration engines can maintain high-throughput synchronization without being weighed down by rigid, heavy cluster managers.

2.5 Theoretical Limits of Distributed Computing

When evaluating distributed processing engines, it is necessary to establish the theoretical boundaries of parallel execution. The efficacy of paradigms like MapReduce is strictly governed by physical network limits and mathematical constraints, most notably Amdahl's Law.

2.5.1 Amdahl's Law and the Sequential Bottleneck

Amdahl's Law formulates the maximum theoretical speedup of a task when executed across multiple parallel processors [6]. Let p represent the proportion of the task that can be strictly parallelized, and N represent the number of active compute nodes. The total system speedup $S(N)$ is defined as:

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}}$$

As the number of nodes N approaches infinity, the maximum theoretical speedup converges to $\frac{1}{1-p}$. This mathematical limit proves that the performance of any distributed system is ultimately gated by its sequential, unparallelizable operations.

In the context of MapReduce engines, the Map and Reduce execution phases are highly parallelizable (increasing p) [1]. However, the intermediate Network Shuffle phase where nodes must halt computation to group and route data acts as a rigid sequential barrier. Modern distributed systems must therefore focus heavily on optimizing network transport and serialization protocols to minimize the duration of this sequential fraction.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Problem Statement

Design and implement a lightweight distributed compute engine that orchestrates Directed Acyclic Graph (DAG) workflows and MapReduce tasks across a network of heterogeneous nodes. In this project, a decoupled Master-Worker architecture is built entirely in Go to solve the operational complexities of traditional cluster management. The system utilizes bounded-memory gRPC streams for high-throughput communication, native OS-level piping for polyglot task execution, and a Write-Ahead Log (WAL) for deterministic state recovery. By eliminating heavy Java Virtual Machine (JVM) middleware and rigid cluster dependencies, this work aims to democratize distributed computing, providing an efficient, language-agnostic framework capable of scaling on standard commodity hardware.

3.2 Problem Description and System Requirements

To effectively reach our goal, the proposed system must successfully abstract the complexities of network routing and state-management away from the end-user. The problem fundamentally reduces to designing a strict, predictable and easy-to-use interface where users can define tasks, task dependencies, and data contracts effortlessly.

3.2.1 Workflow Formalization (The DAG Requirement)

Users need a way to define complex distributed jobs without writing actual network routing code. The system requires a declarative interface where execution workflows are strictly modeled as a Directed Acyclic Graph (DAG), defined mathematically as $G = (V, E)$:

- **Vertices (V):** Each vertex represents an isolated compute task. The system must allow users to define a vertex simply by providing the path to an executable script (like a Python file or a compiled C++ binary) and its necessary arguments.
- **Edges (E):** The edges represent execution barriers and data flow. If there is a directed edge from task A to task B , the system must guarantee that task B will never start until task A is completely finished and its output is secured.

3.2.2 Universal Data Boundaries

To support multiple programming languages without requiring framework-specific SDKs, the system must define a universal data boundary. The interface between the orchestration engine and the user's task must adhere to the following constraints:

- **Ingress Channel (Input):** The system must provide a standardized mechanism to stream unbounded datasets into the task. The expected data format must be predictable and sequentially parsable (such as line-delimited JSON or binary) to ensure the user's code can process incoming data without exhausting local memory.
- **Egress Channel (Output):** There must be a designated, language-agnostic channel for tasks to emit their computed results. The system requires these results to be continuously captured and safely routed to the next phase of the pipeline without requiring explicit network calls from the user's script.
- **Fault and Telemetry Channel:** The system must enforce an isolated boundary strictly reserved for error reporting and telemetry. It requires a deterministic way for a task to signal execution failures, ensuring the orchestrator can immediately recognize the fault, halt the data flow, and prevent the propagation of corrupted intermediate states.

3.2.3 Execution Constraints (Idempotency)

Because distributed hardware is volatile, worker nodes can crash or drop off the network at any time. To recover from this, the system will eventually need to reassign interrupted tasks to new machines.

Because of this necessary retry mechanism, we must enforce a strict requirement on the user's side: all tasks must be idempotent. A script cannot rely on local file system

states or cross-invocation memory. If a task is fed the same input data, it must produce the exact same output data, regardless of how many times the orchestrator is forced to restart it due to hardware faults.

3.2.4 Deployment Requirements

Finally, to solve the friction in traditional enterprise tools, the deployment must be simple.

- The system must not require huge cluster managers or heavy dependencies like the Java Virtual Machine.
- It must be deployable as a standalone binary, where an operator can designate a machine as either the control plane (master) or the compute plane (worker) via simple command-line arguments.
- Worker nodes must be able to discover and connect to the master node dynamically at runtime without needing complex pre-configured routing tables.

3.3 Background and Theoretical Foundations

Before detailing the physical architecture of the proposed system, it is essential to establish the theoretical framework that dictates how data is processed, structured, and transmitted across a distributed network. The Map-Reduce computing paradigm is deeply rooted in functional programming, borrowing heavily from the `map` and `fold` (or `reduce`) primitives found in languages like Lisp and Haskell. By the very nature of these *functions* (in both mathematical and computing sense) on how they mutate the data, the model allows orchestrators to parallelize workloads across thousands of machines safely.

3.3.1 Pure Functions and Side Effects

Pure functions are an important concept in data-processing, a function is called a pure function if it has these qualities:-

- **Deterministic Output:** For a given input x , the function $f(x)$ will always produce the exact same output y , regardless of when or where it is executed. The output doesn't

depend upon anything other than the input, *impure-functions* may differ from this as they may depend upon some kind of other states (globals, hardware, controls etc.).

- **No side-effects:** Side effect is any change that is not in the scope of the function itself. A pure function cannot mutate any outside variable, state, or element.

This means a pure function neither depends upon, nor changes an external state. In a distributed environment, where hardware volatility is a certainty, if a worker node crashes mid-execution, the orchestrator must kill the interrupted task and restart it on a healthy node. If the task contained side effects (e.g., it incremented a shared global counter or partially overwrote a database before crashing), restarting it would corrupt the global state. By strictly modeling the Map and Reduce phases as pure functions, the proposed engine achieves true idempotency. A failed task can be safely abandoned and re-run infinitely until success, guaranteeing exactly-once processing semantics without complex state-rollback mechanisms.

3.3.2 Mathematical Formulation of the Paradigm

The MapReduce model processes data fundamentally as a list of key-value pairs. We can formally define the execution pipeline by observing the domain and codomain of its three distinct phases: **Map, Shuffle, and Reduce**.

1. **The Map Function:** The user-defined Map function takes a single input key-value pair and yields a list of intermediate key-value pairs. Mathematically, if K represents the set of keys and V represents the set of values, the Map signature is:

$$\text{Map} : (K_1 \times V_1) \rightarrow \text{List}(K_2 \times V_2)$$

Because the Map function is pure and only operates on a single (K_1, V_1) pair at a time, the orchestrator can partition the input dataset into arbitrary chunks and apply the Map function concurrently across isolated worker nodes. This phase exhibits trivial $O(N)$ horizontal scalability.

2. **The Shuffle and Group-by-key Barrier:** Once the Map phase concludes, the system encounters an implicit synchronization barrier. The raw intermediate pairs must be routed and aggregated so that all values sharing the same K_2 are grouped together. This system-level operation is modeled as:

$$\text{Shuffle} : \text{List}(K_2 \times V_2) \rightarrow \text{List}(K_2 \times \text{List}(V_2))$$

This phase is traditionally the most expensive operation in a distributed network, requiring all-to-all communication to properly route the keys to their designated reducers.

3. **The Reduce Function:** Following the shuffle, the user-defined Reduce function is invoked. It takes a unique intermediate key K_2 and its associated list of values, performing an aggregation (or fold) to produce a final deterministic output:

$$\text{Reduce} : (K_2 \times \text{List}(V_2)) \rightarrow \text{List}(V_3)$$

By isolating the stateful aggregation strictly to the Reduce phase, the orchestrator ensures that

3.3.3 Theoretical Execution Flow

Figure 1 illustrates this functional transformation pipeline. By representing the computation mathematically, the orchestrator does not need to understand the underlying logic of the user's code; it only needs to fulfill the contract of the data signatures.

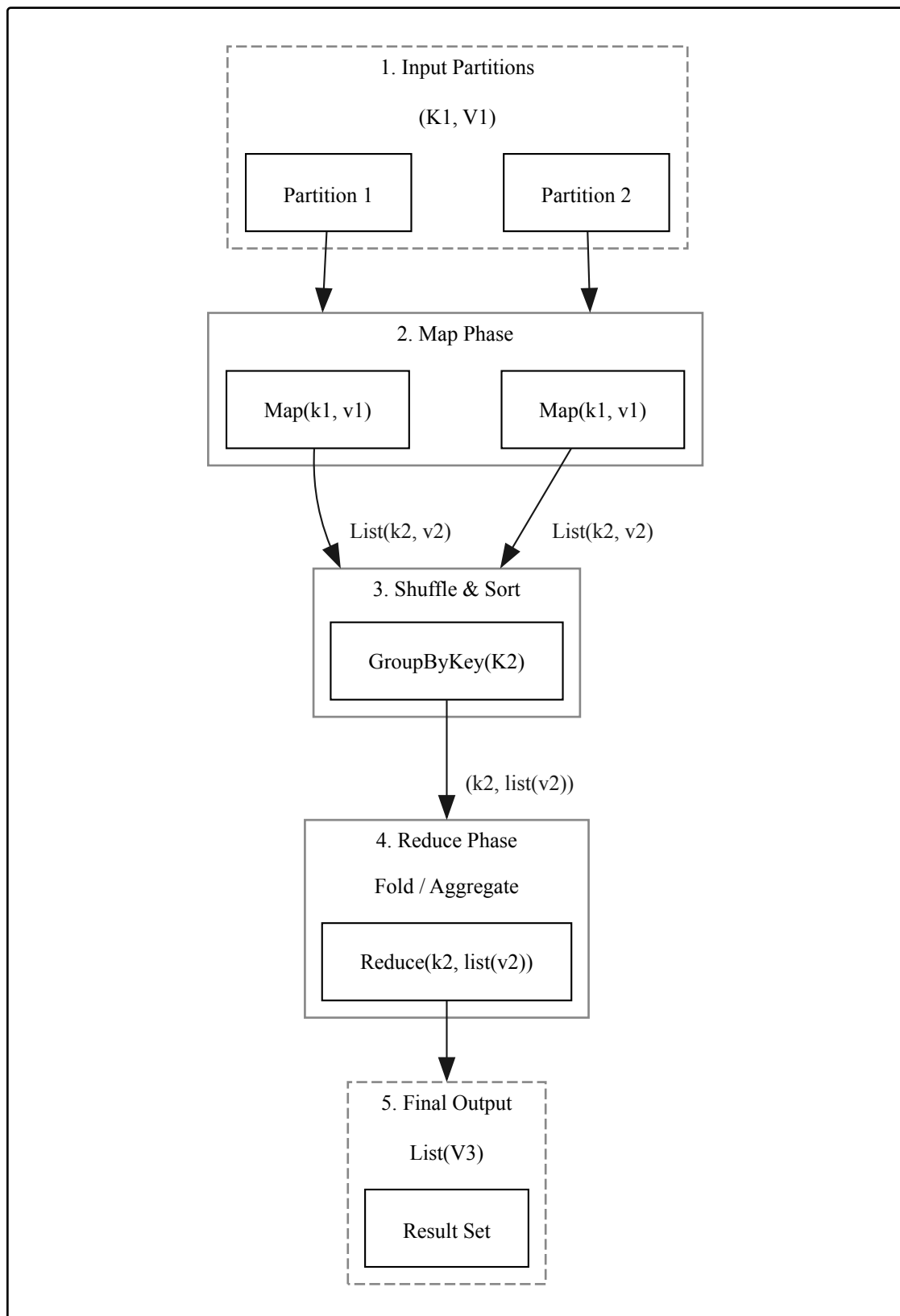


Figure 1: Theoretical data flow and functional boundaries of the MapReduce paradigm

3.3.4 Applied Example: Distributed Word Frequency Analysis

To contextualize the mathematical formulations established above, it is useful to trace a practical data workload through the functional pipeline. The canonical demonstration of the MapReduce paradigm is distributed word frequency analysis (Word Count), where the system computes the occurrence of every word across a massive, unstructured text corpus.

Assume the Central Store holds a dataset that the orchestrator partitions and feeds into two separate Worker nodes. The data transforms through the pure functional boundaries as follows:

1. **Input Partitioning:** The raw input dataset is divided into two bounded chunks and piped to the workers:
 - Partition A: "hello world hello"
 - Partition B: "hello distributed compute"
2. **The Map Phase:** The user-defined Map script reads the input stream. For every word encountered, it emits an intermediate key-value pair where the word is the key and the integer 1 is the value. Because this is a pure function, Node 1 and Node 2 process their chunks in complete isolation without locking a shared database.
 - Node 1 (Processing Partition A) emits:
("hello", 1), ("world", 1), ("hello", 1)
 - Node 2 (Processing Partition B) emits:
("hello", 1), ("distributed", 1), ("compute", 1)
3. **The Shuffle and Group Barrier:** The system intercepts the raw emitted pairs and performs the all-to-all network routing barrier. It groups all identical keys together into lists, ensuring that all occurrences of a specific word are sent to the same Reducer.

Grouped Output:

- "hello" → [1, 1, 1]
- "world" → [1]
- "distributed" → [1]

- "compute" $\rightarrow$ [1]

4. The Reduce Phase:

The user-defined Reduce script receives the grouped keys and their associated lists of integers. It applies a simple aggregation function (a sum fold) to compute the final deterministic totals.

- `Reduce("hello", [1, 1, 1])  $\rightarrow$  ("hello", 3)`
- `Reduce("world", [1])  $\rightarrow$  ("world", 1)`
- `Reduce("distributed", [1])  $\rightarrow$  ("distributed", 1)`
- `Reduce("compute", [1])  $\rightarrow$  ("compute", 1)`

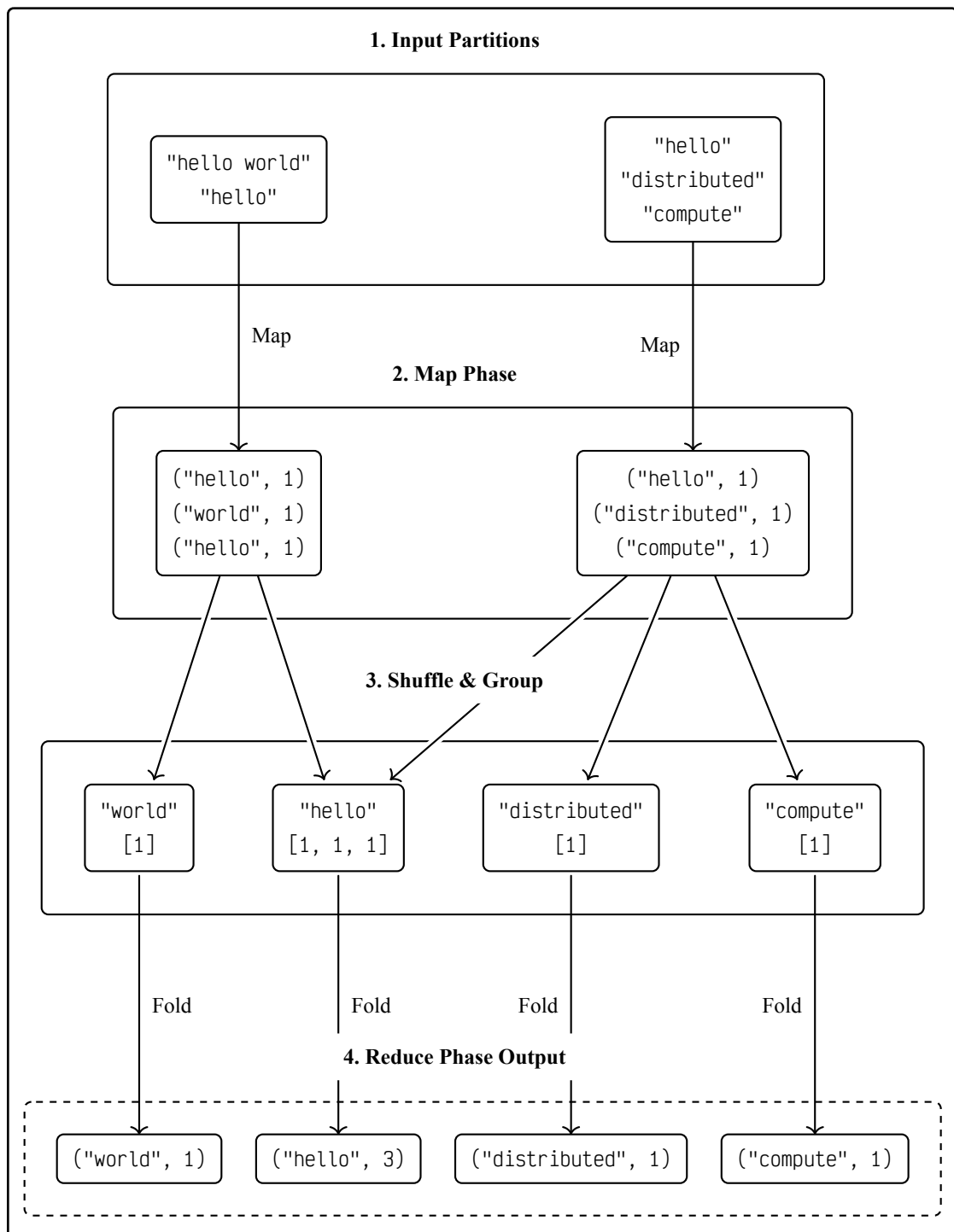


Figure 2: Execution trace of a distributed Word Count workload. Network shuffle intersections are rendered cleanly using crossing halos.

By isolating the counting logic into these stateless phases, the engine natively parallelizes the workload. The developer writing the logic never has to manage race conditions, implement mutex locks, or write network routing code to safely update a shared global counter.

3.3.5 Graph Theory and Directed Acyclic Graphs (DAG)

In computer science and mathematics, complex computational workflows are formally modeled using Graph Theory, specifically as Directed Acyclic Graphs (DAGs). A DAG is defined mathematically as $G = (V, E)$:

- **Vertices (V):** Each vertex represents a distinct computational task or processing stage.
- **Edges (E):** Directed edges represent strict dependencies or data flow requirements. A directed edge $(u, v) \in E$ signifies that task v cannot begin until task u has successfully terminated and yielded its results.

The term “acyclic” dictates that the graph must contain no directed cycles; it is impossible to start at node v and follow a consistently directed sequence of edges that loops back to v . This property is mathematically fundamental for computational pipelines, as a cyclic dependency would inherently represent an infinite deadlock where processes wait on each other endlessly.

3.3.6 Topological Sorting

To execute a DAG on computational hardware, the graph must be mathematically flattened into a one-dimensional, linear sequence that respects all defined dependencies. This is achieved through an algorithmic process known as Topological Sorting.

Input: A Directed Acyclic Graph $G = (V, E)$

Output: A linear list L containing the topologically sorted vertices

```

1:  $L \leftarrow$  Empty list that will contain the sorted elements
2: InDegree  $\leftarrow$  Array storing the number of incoming edges for each  $v \in V$ 
3:  $Q \leftarrow$  Queue containing all nodes where  $\text{InDegree}[v] == 0$ 
4:
5: while  $Q$  is not empty do
6:    $u \leftarrow$  Dequeue a node from  $Q$ 
7:   Append  $u$  to  $L$ 
8:
9:   for each node  $v$  with a directed edge from  $u$  to  $v$  do
10:      $\text{InDegree}[v] \leftarrow \text{InDegree}[v] - 1$  // Relax the edge
11:     if  $\text{InDegree}[v] == 0$  then
12:       Enqueue  $v$  into  $Q$ 
13:
14: if length of  $L \neq |V|$  then
15:   return Error (Graph contains at least one cycle)
16: else
17:   return  $L$ 

```

Figure 3: Pseudocode representation of topological graph resolution via Kahn's Algorithm

A topological sort of a DAG is a linear ordering of its vertices such that for every directed edge (u, v) from vertex u to vertex v , u comes before v in the ordering. This mathematical operation guarantees that no computational stage is scheduled before its prerequisite data has been fully computed. When applied to distributed systems, topological sorting allows independent vertices to be grouped into parallel execution

phases, naturally isolated by execution barriers. This ensures that all intermediate data is completely settled and immutable before subsequent dependent operations begin.

3.3.7 Network Transport Theory: RPC and HTTP/2 Multiplexing

The Remote Procedure Call (RPC) model was originally introduced to allow distributed programs to communicate across a network as seamlessly as local function calls [4]. However, traditional REST APIs and early RPC implementations operating over the HTTP/1.1 protocol are severely bottlenecked by TCP Head-of-Line (HoL) blocking. In HTTP/1.1, multiple requests sent over a single connection must wait in a sequential queue; if one packet is delayed by network congestion, the entire pipeline stalls.

Modern network architectures resolve this by utilizing protocols operating exclusively over HTTP/2 [5]. HTTP/2 introduces binary framing and stream multiplexing, allowing multiple independent RPC messages to be interleaved concurrently over a single, persistent TCP connection. This eliminates HoL blocking, minimizes TCP handshake latency, and provides the highly responsive, bi-directional communication required for continuous telemetry and distributed state synchronization.

3.4 System Architecture & Design

To satisfy the operation requirements laid out in problem statement and requirements sections, the proposed system adopts a Master-Slave architecture developed in Golang. This design separates the control pane, responsible for global state & scheduling/rescheduling from workers which are solely responsible for running tasks.

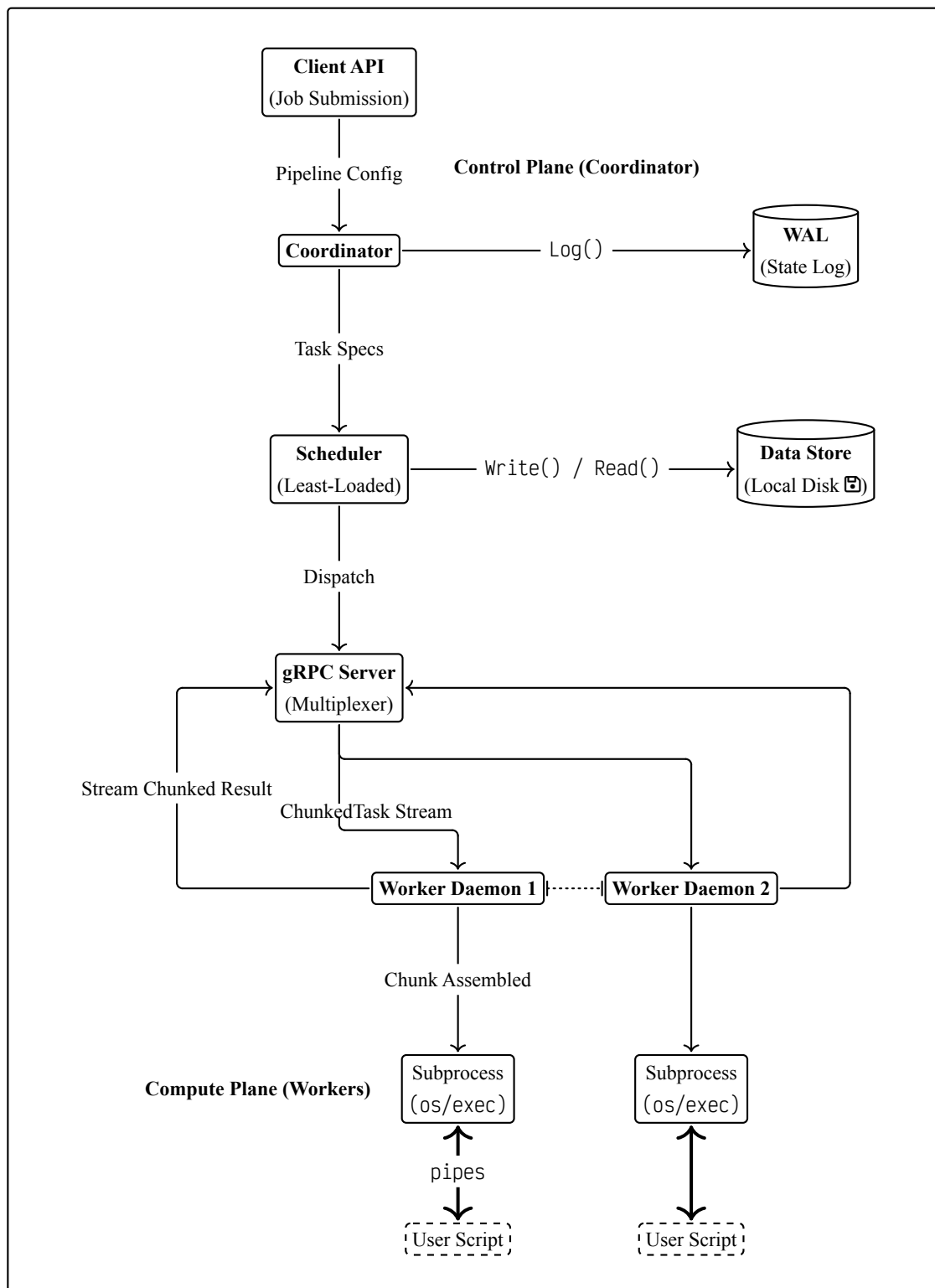


Figure 4: Vertical topology of the Rivage Engine. Both the orchestration layer and the worker subprocess pipelines flow perfectly top-to-bottom.

The architecture outlined in Figure 4 is not merely a structural topology, but a strict sequence of data transformations. To detail the system design, it is best to track the execution lifecycle of a dataset as it moves through the distinct physical boundaries of the network. The orchestrator must handle ingestion, static topology resolution, bounded data streaming, etc.

3.4.1 The Control Plane and DAG Resolution

The execution lifecycle begins at the Coordinator, which serves as the central control plane for the cluster. It is isolated from all user-space computation.

When a user submits a MapReduce job via the API, the payload defines the logical workflow using the `dag.Builder` interface. The Coordinator routes this payload to the internal DAG parser, which applies Kahn’s algorithm to perform a static topological sort. This mathematically flattens the multi-stage dependency graph into a linear, acyclic execution array (`pipeline.Order`).

For the initial pipeline stage (the source stage), the Coordinator partitions the raw dataset into discrete `TaskInput` structures. Each fragment is wrapped in a `Protocol Buffer TaskSpec` contract. This contract dictates the target executable command, required environment variables, and specific hardware tags needed for execution.

3.4.2 State Persistence and the Write-Ahead Log (WAL)

Because the Coordinator manages the execution state of the entire cluster in volatile memory, it represents a single point of failure. If the master node loses power, the execution graph is destroyed.

To achieve fault tolerance without the latency of external database transactions, the system implements a localized Write-Ahead Logging (WAL) mechanism. Before any `TaskSpec` is routed over the network, the Coordinator generates a unique session ID and appends a deterministic state transition event (e.g., `walEventDispatched`) to a binary `.wal` file on the local disk. In parallel, it initializes a directory in the `ResultStore`, a disk-backed key-value store designed to hold intermediate computational outputs. If the master crashes, the `RecoverEntries` routine replays this log to reconstruct the exact graph state, bypassing previously completed tasks and re-queuing pending ones.

3.4.3 Scheduler Allocation and Optimistic Locking

Pending tasks are picked up by the asynchronous `watchJob` scheduling routine. The Scheduler evaluates the active Compute Plane using a set of real-time worker telemetry snapshots.

To prevent network race conditions—where the master might spam tasks to a single worker before its CPU telemetry updates—the Scheduler uses an optimistic locking approach. The Coordinator maintains a concurrent map of active workers, tracking their reported load alongside a local `inFlight` atomic integer. The Scheduler selects a worker based on either a Least-Loaded or Tag-Affinity algorithm, factoring in the `inFlight` variable to guarantee that node capacities are never over-subscribed.

3.4.4 Network Transport and Bounded-Memory Chunking

Once a worker is allocated, the Coordinator initiates network transfer. Sending monolithic data payloads (such as a 2GB text file or massive matrix multiplication tiles) across standard RPC frameworks immediately triggers Out-Of-Memory (OOM) kernel panics. To bound memory complexity to $O(1)$, the engine utilizes a custom chunking pipeline over HTTP/2 multiplexed gRPC streams. The Coordinator fragments the task payload into strict 4MB segments. These segments are pushed asynchronously to the target worker as `ChunkedTask` streams.

On the receiving end, the Worker Daemon catches the stream using an `inboundAssembler`. Rather than accumulating the 4MB slices in RAM, the assembler writes the incoming byte stream directly to a temporary file on the worker's local storage. This mechanism allows workers with standard commodity hardware to process datasets infinitely larger than their available system memory.

3.4.5 Polyglot Execution via POSIX Boundaries

Once the binary stream is fully assembled on the worker's disk, the data crosses the execution boundary. The Worker Daemon acts as a purely agnostic vessel; it does not parse or understand the data it holds.

To execute the task, the daemon leverages Go's `os/exec` library to spawn an isolated operating system subprocess running the user's defined script (e.g., a Python interpreter

or a C++ binary). The daemon passes the path of the assembled data file to the script via the `RIVAGE_INPUT_FILE` environment variable, or natively pipes it into the process's standard input (`stdin`).

The user's script performs the data transformation natively. It outputs its deterministic result to standard output (`stdout`), which the Worker Daemon simultaneously captures and redirects to a new local output file.

3.4.6 Upstream Telemetry and Stage Synchronization

When the subprocess exits, the Worker Daemon must return the data to the master. Utilizing a `transfer.ChunkReader`, the worker slices the new output file into 512KB frames and streams them upward via `ChunkedResult` gRPC messages.

The Coordinator intercepts these frames and pipes them directly into the durable `ResultStore` established in step two. It then logs a `walEventCompleted` entry. Once the worker finishes transmitting, it aggressively purges its local temporary files, returning to a stateless, idle condition ready for the next dispatch loop.

The Coordinator strictly enforces stage synchronization barriers. It halts graph progression until all tasks within the current topological stage successfully report completion. Once the barrier is cleared, the Scheduler extracts the aggregated outputs from the `ResultStore`, passes them through the user's defined mathematical routing logic (the `ShuffleFunc`), and uses the grouped output to generate tasks for the subsequent DAG stage.

3.5 The Coordinator (Control Plane)

The Coordinator serves as the master node and central nervous system of the Rivage architecture. Operating entirely outside the user-space computation layer, it acts as a deterministic finite state machine, network multiplexer, and resource allocator. Because the Coordinator manages the volatile state of the entire distributed cluster, its internal architecture is heavily optimized to bypass memory saturation and thread starvation.

3.5.1 Core Responsibilities

The Coordinator isolates cluster orchestration from data processing. Its operational directives are strictly confined to the following:

- **DAG Orchestration and Topological Sorting:** Ingests logical `dag.Pipeline` definitions from the Client API. It executes Kahn’s Algorithm to flatten the multi-dimensional dependency graph into a linear array of execution stages, guaranteeing that downstream tasks are structurally blocked until upstream prerequisites resolve.
- **Task Generation and Payload Fragmentation:** Scans raw input datasets (such as a massive log file or binary stream) and mathematically partitions them into bounded fragments. It binds these fragments to the user’s defined executable scripts, yielding independent execution payloads.
- **State Persistence and Disaster Recovery:** Circumvents the need for heavy external databases by committing every pipeline state mutation to an append-only Write-Ahead Log (WAL). Combined with a localized `ResultStore` for intermediate data, this ensures deterministic recovery after a master node kernel panic.
- **Scheduler Delegation:** Continuously polls the internal state machine. It delegates hardware routing to the Scheduler interface, utilizing an optimistic locking algorithm to prevent network congestion and hardware over-subscription.
- **gRPC Multiplexing:** Maintains persistent, bi-directional HTTP/2 TCP streams with every authenticated worker daemon in the cluster, facilitating simultaneous payload dispatching and real-time hardware telemetry ingestion.

3.5.2 Initialization and Network Configuration

Upon boot, the Coordinator provisions a high-performance gRPC server. Standard gRPC configurations utilize conservative memory limits that are entirely unsuitable for batch processing. The Coordinator explicitly overrides the underlying transport constraints during startup:

- **Payload Boundaries:** `grpc.MaxRecvMsgSize` and `grpc.MaxSendMsgSize` are statically elevated to 16 Megabytes to accommodate large data payloads without triggering transport panics.

- **TCP Windowing:** Transport buffers (`grpc.WriteBufferSize` and `grpc.ReadBufferSize`) are expanded to 8 Megabytes, preventing back-pressure during highly concurrent chunk streaming over local networks.

Concurrently, the Coordinator provisions an asynchronous HTTP server acting as an administrative dashboard. It intercepts standard operational logs via a custom `telemetry.CaptureWriter`, injecting them into a thread-safe circular ring buffer (`LogRing`). This allows the HTTP endpoint `/api/state` to serve real-time cluster telemetry, including worker CPU load and global pipeline metrics, directly to a web interface without disrupting the core gRPC event loop.

To ensure cluster security, especially in zero-trust environments, the Coordinator instantiates a `security.TokenSigner` utilizing Mutual TLS (mTLS). When a Worker Daemon attempts to register, the Coordinator mathematically verifies its cryptographically signed HMAC-SHA256 token before admitting the TCP connection into the active worker pool.

3.5.3 Internal Struct Layout and Concurrency Mechanics

Managing thousands of concurrent network streams demands strict memory partitioning. A monolithic global read-write lock would immediately throttle the HTTP/2 event loop. Therefore, the Coordinator fragments its state across lock-free concurrent maps and localized mutexes.

3.5.3.1 The Global Root

The root struct maintains the immutable configuration boundaries, network interfaces, and the global thread-safe registries. By mapping identifiers directly to memory pointers within a concurrent map, asynchronous goroutines can fetch a specific worker's state or a job's status in $O(1)$ time complexity.

```
struct Coordinator {  
    // Interfaces  
    cfg      : CoordinatorConfig  
    sched    : SchedulerInterface  
    grpc_srv : ProtocolServer  
    log      : TelemetryLogger  
  
    // Global Registries  
    workers : ConcurrentMap [WorkerID → worker_state]  
    jobs    : ConcurrentMap [JobID → job_state]  
    results : ConcurrentMap [JobID → OutputCache]  
}
```

3.5.3.2 Worker State (Telemetry & Networking)

The `worker_state` tracks the networking footprint of a single daemon. To accommodate high-frequency heartbeat packets without stalling the network reader, it aggressively utilizes hardware-level atomic operations.

```
struct worker_state {  
    // Locks  
    mu          : ReadWriteLock // Protects heartbeat timestamp  
    send_mu     : StandardLock  // Serializes outbound gRPC  
    frames  
  
    // Identity  
    id          : String  
    stream      : GrpcConnection  
    last_heartbeat : Timestamp  
  
    // Hardware Telemetry  
    active_tasks : AtomicInteger  
    in_flight    : AtomicInteger  
    cpu_usage    : AtomicFloat  
}
```

Architectural Justification: By separating `mu` from `send_mu`, the system achieves true I/O multiplexing. The `mu` lock exclusively guards the `last_heartbeat` timestamp. Metrics

like `cpu_usage` are mutated via lock-free atomic swap instructions. Consequently, the Coordinator can ingest a telemetry frame, update the internal CPU gauge, and dispatch a 4MB data chunk to the same worker simultaneously, completely devoid of cross-thread blocking.

3.5.3.3 Job and Task State

A job encapsulates a single execution stage (e.g., the Shuffle phase). The `task_state` tracks the absolute lowest-level execution unit. State mutations lock only the specific `job.mu`, keeping operations granular and localized.

```
struct job {
    mu                : StandardLock
    tasks             : Map [TaskID → task_state]
    done_tasks        : AtomicInteger

    // Disk Boundaries
    store             : ResultStorePointer
    wal               : WriteAheadLogPointer
    chunk_assemblers  : Map [TaskID → ByteStream]
}

struct task_state {
    spec             : TaskSpecification
    status           : Enum (Pending | Running | Completed | Failed)
    worker_id        : String
    retries          : Integer
}
```

3.5.4 Dynamic Code Shipping and Protocol Buffers

To eliminate the necessity of a shared distributed file system (e.g., NFS), the Coordinator implements a dynamic `codeCache`. Before execution begins, it scans the DAG for `CodeFile` declarations. It reads the source code from the Master's local disk, caches the raw byte array in memory, and embeds it directly into the outbound `pb.TaskSpec` protocol buffers. The Coordinator physically ships the execution logic alongside the dataset over the network, ensuring workers require zero pre-configuration.

The Coordinator dispatches two primary payload types over the gRPC stream:

- TaskSpec: Utilized when the data partition fits safely within memory boundaries. It encapsulates the binary data, POSIX environment variables, and the embedded source code script.
- ChunkedTaskSpec: Invoked when the input data breaches the safe dispatch threshold (4MB). The Coordinator sequentially fragments the byte array, dispatching indexed frames. This bounded-memory streaming eliminates Head-of-Line (HoL) blocking and prevents the master node’s RAM from saturating during massive dataset dispatches.

3.5.5 Task Lifecycle and WAL Recovery

3.5.5.1 Logical Definition

A “Task” acts as the atomic unit of distributed compute. Logically, it binds a discrete data partition to a defined execution script within an isolated OS environment. Because user-defined scripts are mandated to act as idempotent pure functions (producing deterministic outputs without mutating global variables), the Coordinator can safely terminate, replicate, and re-assign tasks unconditionally upon hardware failure.

3.5.5.2 Execution Lifecycle (Finite State Machine)

The execution of a task is governed by a strict finite state machine tracked within the `job.tasks` mapping:

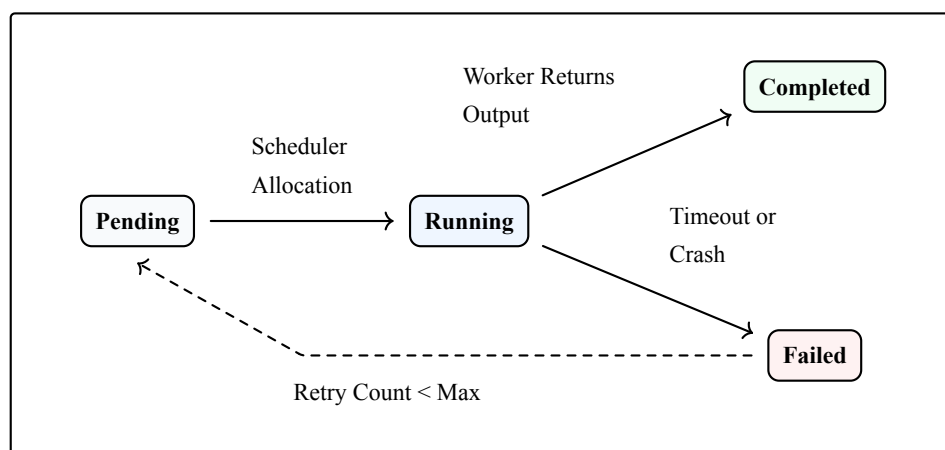


Figure 5: The internal task state machine managed by the Coordinator.

1. **Pending (and Recovery):** When a stage begins, the Coordinator opens the binary Write-Ahead Log (.wal). It scans the log to reconstruct state. Tasks flagged as `walEventCompleted` with valid outputs on disk are preemptively marked as **Completed**, skipping dispatch. Unfinished tasks are initialized as **Pending**.
2. **Running:** The Scheduler successfully acquires a worker. The worker's `in_flight` atomic integer is optimistically incremented, a `walEventDispatched` record is logged, and the data is piped into the gRPC stream.
3. **Completed:** The worker node streams the computed output back to the Coordinator, triggering a disk write to the `ResultStore` and a terminal state update.
4. **Failed:** The worker subprocess panics (yielding a non-zero exit code), or the global watchdog intercepts a node timeout. If the retry threshold is not breached, the state reverts to **Pending** for immediate re-allocation.

3.5.6 Critical Function Signatures

The internal state mutations and network I/O boundaries are abstracted behind a core set of highly optimized routines:

```
// Primary entry point for job ingestion and topology parsing
run_job_raw(ctx Context, job_id String, pipeline Pipeline,
            inputs []TaskInput) → ([]TaskOutput, Error)

// Stage Executor (Invoked iteratively for pipeline.Order)
execute_stage(ctx Context, job_id String, stage Stage,
              tasks []TaskSpec) → ([]TaskOutput, Error)

// Network Dispatcher (Handles Chunking Logic)
dispatch(worker_id String, spec TaskSpec) → Error
```

3.5.7 Orchestration Algorithms

To maintain the integrity of the state machine without stalling the server, the Coordinator executes two primary asynchronous loops: the task driver and the hardware watchdog.

3.5.7.1 The Task Driver (`watchJob`)

Instead of reacting to every incoming network event (which can trigger CPU-intensive broadcast storms), the Coordinator utilizes a batched, interval-based driver routine per stage.

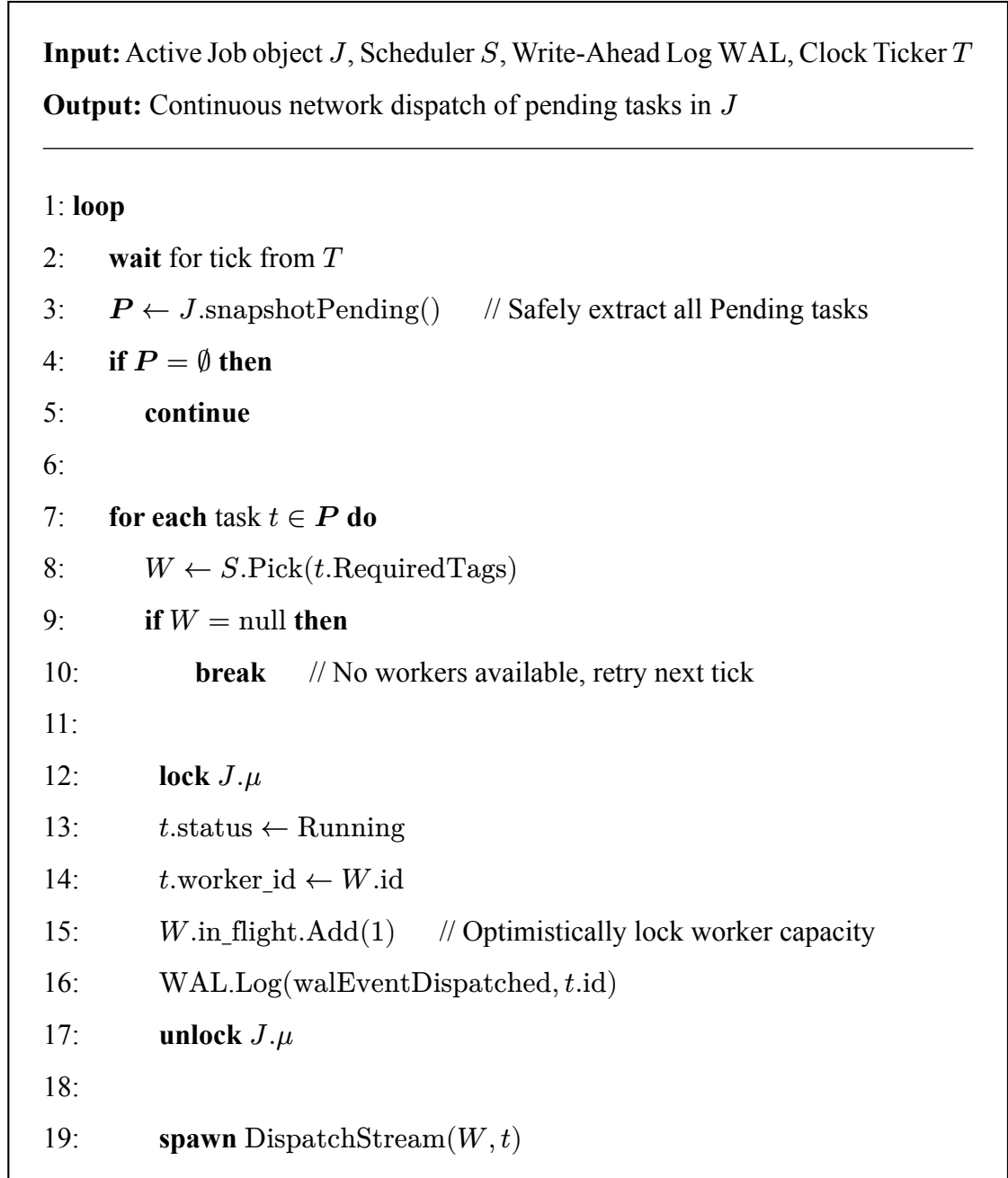


Figure 6: Pseudocode representation of the asynchronous batched state evaluation loop
(`watchJob`)

3.5.7.2 Hardware Watchdog (`reapDeadWorkerTasks`)

Because worker daemons run on volatile hardware, the cluster must deterministically identify silent crashes or network partitions. A global watchdog routine continuously evaluates the `last_heartbeat` timestamps of all registered nodes.

If the delta between the system clock and the last heartbeat exceeds the tolerance threshold, the Coordinator formally declares the node dead. The reaper algorithm intervenes directly in the `job.tasks` state machine: it downgrades any task assigned to the dead worker from `Running` to `Pending` and clears the `worker_id`. On the next tick of the `watchJob` algorithm, the pending task is seamlessly re-routed to a healthy node, ensuring exactly-once processing semantics without user intervention.

3.5.8 Extended Responsibilities: Telemetry and Assembly

Beyond active dispatching and scheduling, the Coordinator handles two critical passive operations:

1. **Lock-Free Heartbeat Ingestion:** It processes a continuous barrage of `WorkerMessage_Heartbeat` frames. By updating `atomic.Value` fields in the `worker_state`, it provides the Scheduler with real-time CPU utilization metrics without requiring mutex locks, preventing bottlenecks at the network boundary.
2. **Disk-Backed Byte Stream Assembly:** When workers compute large outputs, they return the data via `ChunkedResult` streams. The Coordinator buffers these incoming frames using a temporary `chunk_assembler`. Upon receiving the EOF (`IsFinal`) flag, the assembled byte array is flushed directly to the disk-backed `ResultStore`. This mechanism completely shields the master node's volatile memory pool, enabling the system to aggregate gigabytes of intermediate data safely.

3.6 The Scheduler (Resource Allocation and Load Balancing)

In a distributed computing cluster, blindly dispatching tasks to worker nodes inevitably leads to hardware saturation, severe network bottlenecks, and uneven execution times. This phenomenon, formalized in distributed systems literature as the “straggler problem,” occurs when a subset of worker nodes receives more computational load than their CPU cores can process, while other nodes sit idle. Because the overall execution time

of a Directed Acyclic Graph (DAG) stage is strictly bounded by the completion time of its slowest task, optimal load balancing is a critical requirement for cluster throughput. To mitigate this, the Rivage engine strictly decouples network dispatching logic from hardware allocation logic. As established in the previous section, the Coordinator manages the execution state and network multiplexing, but it does not make routing decisions. When the asynchronous driver identifies a pending task, it pauses the dispatch pipeline and delegates the hardware routing entirely to the Scheduler subsystem.

3.6.1 Core Responsibilities and Theoretical Challenges

The Scheduler operates as a synchronous, deterministic routing engine. The fundamental problem it solves maps closely to the classic multi-processor scheduling problem (a variation of the Bin Packing Problem), which is mathematically NP-hard. Because an exact optimal routing would require perfect future knowledge of task execution times—which is impossible for arbitrary user-defined scripts—the Scheduler relies on real-time, greedy heuristics.

Its primary operational responsibilities are:

- **Constraint Satisfaction (Hardware Filtering):** Evaluating pending tasks against the active worker pool to immediately disqualify nodes that lack the required execution environment (e.g., filtering out standard compute nodes when a dispatched binary explicitly requires specialized instruction sets like AVX-512 via execution tags).
- **Dynamic Load Balancing:** Analyzing real-time hardware telemetry—specifically CPU utilization and active process counts—to route tasks to the least-burdened nodes in the cluster.
- **Data Locality and Affinity:** Attempting to route tasks to workers that already have necessary intermediate data cached locally on their NVMe drives. This minimizes redundant network transfers across the HTTP/2 streams.
- **Latency Masking:** Utilizing prefetch multipliers to computationally inflate a worker’s capacity limits, intentionally over-subscribing idle network pipes to hide the round-trip time (RTT) latency of the gRPC dispatch sequence.

3.6.2 Interface Design and Memory Isolation

In monolithic distributed systems, the scheduler is often tightly coupled to the master node's central database or state registry. This architectural flaw introduces massive read/write lock contention; the scheduler cannot route a task without locking the state, thereby blocking incoming telemetry from workers.

In Rivage, the Scheduler is implemented as a completely isolated interface. To ensure routing algorithms execute in sub-millisecond time, the Coordinator passes a detached, read-only slice of the cluster state to the Scheduler.

```
interface Scheduler {  
    // Evaluates telemetry and returns the optimal Worker ID  
    Pick(workers: []WorkerSnapshot, req_tags: []String,  
        affinity: []String) → String  
}  
  
// Read-only, lock-free copy of daemon telemetry.  
struct WorkerSnapshot {  
    id          : String  
    tags        : []String  
    active_tasks : Integer  
    cpu_usage   : Float  
    last_heartbeat : Timestamp  
    cached_keys : []String // Local data availability  
    capacity    : Integer  // Hardware core limits  
}
```

Before the Coordinator invokes the `Pick` method, it iterates through its lock-free concurrent map. It constructs an array of `WorkerSnapshot` structs utilizing hardware-level atomic values. By passing this detached array, the Scheduler performs its mathematical scoring matrix entirely in CPU cache memory, avoiding network locks or synchronization blocking.

3.6.3 Algorithmic Implementations

The engine supports multiple routing algorithms, allowing cluster operators to hot-swap scheduling logic depending on the nature of their workload (e.g., homogeneous batch processing vs. heterogeneous microservices).

3.6.3.1 Round-Robin Allocation (The Baseline)

The Round-Robin algorithm serves as the simplest routing heuristic. It maintains an internal pointer to the last assigned worker and increments it sequentially for every new task, wrapping around via a modulo operator when the end of the pool is reached.

While computationally cheap ($O(1)$ time complexity per allocation), it is inherently “blind.” It assumes all tasks take the exact same amount of time and all workers have identical hardware throughput. In a real-world polyglot environment, Round-Robin inevitably leads to hardware starvation and severe straggler effects.

Notice that the `RoundRobinScheduler` requires an internal standard lock to safely increment the `index` variable when the Coordinator’s asynchronous driver goroutines call the algorithm concurrently.

3.6.3.2 Hardware Filtering and Set Theory Constraint Satisfaction

For complex workloads, the Scheduler must respect strict hardware constraints. When a DAG stage requires specific environments, the task payload is annotated with `RequiredTags` (e.g., `["arch=amd64", "memory=high"]`).

The Scheduler treats this constraint satisfaction as a strict mathematical subset problem. Let R represent the set of required tags defined by the task, and let T_w represent the set of tags reported by a specific worker w . The Scheduler evaluates the boolean intersection:

$$R \subseteq T_w$$

Any worker that fails to satisfy the entire required subset is mathematically eliminated from the candidate pool before load scoring even begins. This zero-tolerance filtering ensures that a workload requiring specific architectural features is never dispatched to an incompatible node.

3.6.3.3 The Thundering Herd Problem and Optimistic Locking

The most critical engineering challenge in any distributed scheduler is preventing the “Thundering Herd” phenomenon, caused by the latency gap between the Control Plane and the Compute Plane.

Assume the Coordinator has 100 pending tasks. The Coordinator loops rapidly, asking the Scheduler for a worker. The Scheduler sees that Worker A has 0 active tasks. It assigns Task 1 to Worker A. Milliseconds later, it asks for a worker for Task 2. Because Task 1 is still traveling over the network as a gRPC binary stream, Worker A has not yet updated its telemetry heartbeat. Worker A **still** reports 0 active tasks. Without intervention, the Scheduler will erroneously assign all 100 tasks to Worker A, causing a massive memory fault the moment the network buffers flush.

To solve this, the Rivage engine uses **Optimistic Locking**. The exact moment the Scheduler returns a worker, the Coordinator increments an `in_flight` atomic counter. When generating the `WorkerSnapshot` for the next scheduling iteration, the Coordinator merges this counter directly into the `ActiveTasks` value. This guarantees that the Scheduler perceives the incoming network load instantly, mathematically distributing the tasks across the cluster even if the physical telemetry heartbeats lag behind.

3.6.3.4 Network Latency Masking and Capacity Limits

The physical reality of distributed systems dictates that network latency (Round-Trip Time) is orders of magnitude slower than CPU cycle times. To keep hardware saturated, the Scheduler bounds the available worker pool using a configurable prefetch multiplier (M_p).

The mathematical capacity limit $L_{\max}$ for a given worker w is defined as:

$$L_{\max} = \max(1, \lfloor w.\text{Capacity} \times M_p \rfloor)$$

- **Multiplier 1.0 (Strict Mapping):** The limit strictly matches the core count. This is ideal for heavy, sustained CPU work where context switching must be minimized.
- **Multiplier 2.0+ (Over-Subscription):** The limit is artificially inflated. This allows the Coordinator to dispatch tasks to a worker’s internal queue **before** it has finished its current execution payload.

The Scheduler strictly filters out any worker whose `ActiveTasks` meets or exceeds this $L_{\max}$ limit, effectively applying backpressure to the entire pipeline.

3.6.3.5 Dynamic Data Locality and Affinity Routing

In MapReduce workflows, massive datasets are partitioned. If Worker Node A processed a 10GB dataset during the Map phase, it already has the intermediate output data resident on its local drive. If the subsequent Reduce phase requires that same data, routing the task to Node B would trigger a massive network transfer, crippling cluster throughput.

To handle this, the Scheduler evaluates `AffinityKeys`. Let $\mathbf{A}$ be the set of affinity keys requested by the task. The algorithm calculates an intersection size (the Affinity Score) for all eligible workers:

$$S_w = |w.\text{CachedKeys} \cap \mathbf{A}|$$

The system performs a strict maximization phase. It identifies the maximum affinity score present in the cluster and immediately discards any worker that does not tie for this maximum score. Only after isolating the nodes with the absolute best data locality does the algorithm fall back to load balancing.

3.6.4 Least-Loaded Allocation (The Core Algorithm)

The default routing strategy for the Rivage engine is the Least-Loaded algorithm. It executes an $O(N)$ sweep to filter nodes by tags and capacity, isolates the subset of nodes with the highest data locality, and finally selects the node with the lowest `ActiveTasks`. In the event of a tie, it utilizes the real-time `CPUUsage` telemetry as a secondary tie-breaker.

The exact execution flow of the routing decision is illustrated in Figure 6.

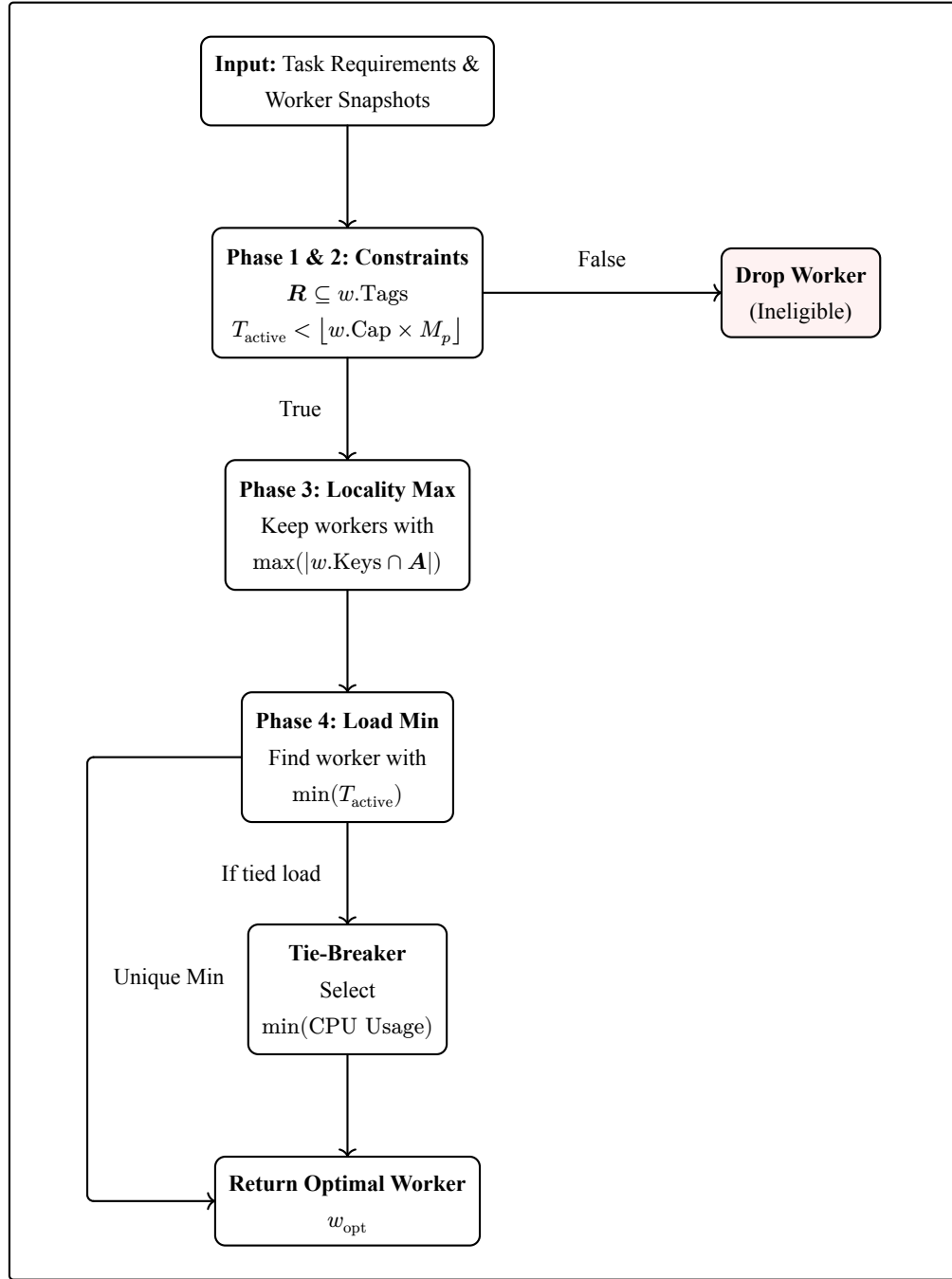


Figure 7: Execution flow and decision matrix of the Least-Loaded routing algorithm.

To formalize the flowchart, the algorithm is implemented sequentially. To prevent visual overflow, it is detailed across two phases: Candidate Formulation (Algorithm 2) and Heuristic Resolution (Algorithm 3).

Input: Required Tags $\mathbf{R}$, Affinity Keys $\mathbf{A}$, Worker snapshots $\mathbf{W}$, Prefetch Multiplier M_p

Output: Localized candidate pool $\mathbf{C}_{\text{local}}$

```

1:  $\mathbf{C} \leftarrow \emptyset$ 
2:
3: // Phase 1 & 2: Hardware Constraints and Capacity Filtering
4: for each worker  $w \in \mathbf{W}$  do
5:    $L_{\text{max}} \leftarrow \max(1, \lfloor w.\text{Capacity} \times M_p \rfloor)$ 
6:   if  $\mathbf{R} \subseteq w.\text{Tags}$  and  $w.\text{ActiveTasks} < L_{\text{max}}$  then
7:      $\mathbf{C} \leftarrow \mathbf{C} \cup \{w\}$ 
8:
9: if  $\mathbf{C} = \emptyset$  then return  $\emptyset$ 
10:
11: // Phase 3: Locality Maximization
12:  $S_{\text{max}} \leftarrow -1$ 
13: for each worker  $w \in \mathbf{C}$  do
14:    $S_w \leftarrow |w.\text{CachedKeys} \cap \mathbf{A}|$ 
15:   if  $S_w > S_{\text{max}}$  then  $S_{\text{max}} \leftarrow S_w$ 
16:
17:  $\mathbf{C}_{\text{local}} \leftarrow \{w \in \mathbf{C} \mid |w.\text{CachedKeys} \cap \mathbf{A}| = S_{\text{max}}\}$ 
18: return  $\mathbf{C}_{\text{local}}$ 

```

Figure 8: Part 1: The $O(N)$ filtering matrix isolating hardware constraints and maximizing data locality.

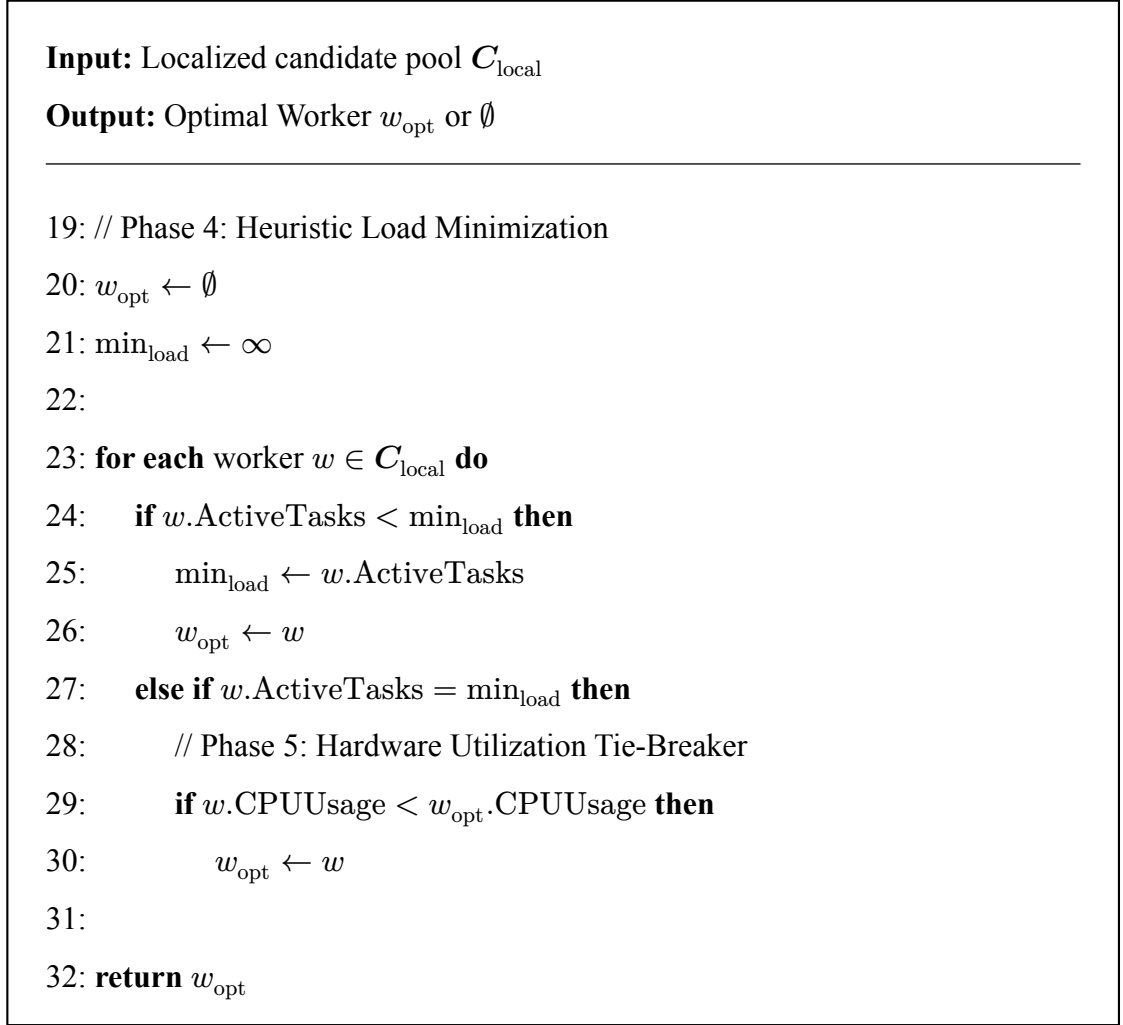


Figure 9: Part 2: Applying the least-loaded heuristic and physical CPU tie-breakers to select the final node.

3.6.5 Algorithmic Complexity Analysis

The architectural decision to detach the `WorkerSnapshot` array from the Coordinator's global `sync.Map` guarantees that the `Pick` algorithm executes purely in CPU cache memory.

For a cluster containing N worker nodes and a task defining K affinity keys:

- The Hardware Constraint and Capacity Filtering phase requires a subset check, operating in $O(N)$ time.
- The Locality Maximization phase iterates through the filtered candidate pool, checking up to K keys, operating in $O(N \times K)$ time.

- The Heuristic Load Minimization phase operates in $O(N)$ time.
- The overall Space Complexity is $O(N)$ to store the temporary candidate pools C and C_{local} .

Because K (the number of affinity keys) is strictly bounded and typically very small (e.g., $K < 5$), the total Time Complexity simplifies asymptotically to $O(N)$. Given modern CPU clock speeds, an $O(N)$ linear scan across an array of a few thousand structs resolves in less than a millisecond, ensuring the Scheduler routing matrix never acts as the bottleneck for cluster throughput.

3.7 The Worker Daemon (Compute Plane)

If the Coordinator acts as the central nervous system of the Rivage architecture and the Scheduler as its routing matrix, the Worker Daemon represents the raw, distributed muscle. Operating entirely within the Compute Plane, the worker is designed as a lightweight, zero-dependency, and language-agnostic execution vessel. It possesses no knowledge of the global Directed Acyclic Graph (DAG) topology, nor does it understand the structural semantics of the data it processes. Its sole operational directive is to securely receive binary streams, execute isolated operating system processes, and return deterministic results.

3.7.1 Core Responsibilities and Execution Philosophy

To maintain the absolute decoupling of orchestration and execution, the Worker Daemon adheres to a strict set of responsibilities:

- **Persistent Connection Maintenance:** Bootstrapping a persistent, multiplexed HTTP/2 gRPC connection back to the Control Plane and securing it via Mutual TLS (mTLS) authentication.
- **Telemetry Broadcasting:** Continuously probing the underlying physical hardware (via OS-level metrics) and asynchronously broadcasting microsecond-accurate CPU saturation and active task loads back to the Coordinator.
- **Bounded-Memory Ingestion:** Reassembling massive partitioned datasets streamed over the network by writing incoming chunks directly to local volatile storage (NVMe/SSD), strictly avoiding Out-Of-Memory (OOM) kernel panics.

- **POSIX Subprocess Isolation:** Spawning isolated execution environments utilizing the host operating system’s standard process forking mechanisms (`os/exec`). This allows the daemon to execute arbitrary polyglot scripts (Python, C++, Rust) without requiring heavy framework-specific SDKs or JVM wrappers.
- **Upstream Result Streaming:** Capturing the standard output (`stdout`) of the executed subprocesses, buffering it, and streaming it back to the Coordinator in controlled 512KB binary frames.

3.7.2 Interface Design and Concurrency Limits

Because a worker daemon operates on physical commodity hardware, it is bound by finite resources (CPU cores and RAM). If the Coordinator dispatches 32 concurrent tasks to a 4-core machine, attempting to spawn 32 simultaneous execution threads would trigger severe context-switching overhead, effectively thrashing the CPU cache and destroying throughput.

To enforce strict hardware boundaries, the Worker Daemon utilizes a bounded concurrency model implemented via a Counting Semaphore.

```
struct WorkerDaemon {
    id          : String
    cfg         : WorkerConfig
    client      : GrpcWorkerServiceClient
    cpu_cores   : Integer
    active_tasks : AtomicInteger
    semaphore   : Channel [Size: cpu_cores]
    task_mu     : StandardLock
    running     : Map [TaskID → active_task]
}

struct active_task {
    spec          : TaskSpecification
    cancel_func   : ContextCancelFunction
    temp_dir      : String
    start_time    : Timestamp
}
```

The semaphore is a buffered channel strictly initialized to the number of physical CPU cores on the host machine (e.g., `runtime.NumCPU()`). Before the daemon spawns an OS-level subprocess, it must acquire a token from this semaphore. If the semaphore is fully saturated, the incoming task is blocked in memory. This mathematically guarantees that the daemon never exceeds a 1:1 active thread-to-core ratio, allowing the physical CPU to execute instructions with maximum cache locality.

3.7.3 Bounded-Memory Data Assembly

As detailed in the Coordinator architecture, sending monolithic data payloads (such as a 2GB log file) across an RPC framework triggers immediate memory exhaustion. To circumvent this, the Coordinator slices payloads into 4MB `ChunkedTaskSpec` frames.

When these frames arrive at the Compute Plane, the Worker Daemon must intercept them without accumulating the bytes in RAM. It achieves this via a temporary file-backed `ChunkAssembler`.

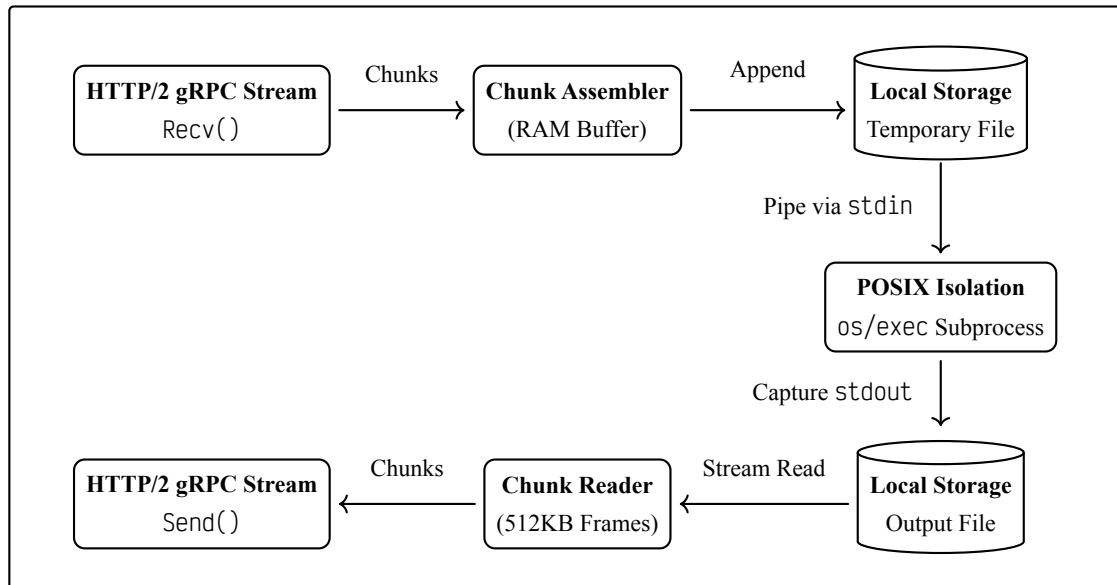


Figure 10: The Worker Daemon’s bounded-memory I/O pipeline. Data is continuously flushed to local disk to preserve RAM during polyglot execution.

By maintaining this strict disk-buffering protocol, a worker node with only 2GB of physical RAM can comfortably process a 50GB dataset partition without crashing.

3.7.4 Polyglot Execution via POSIX Boundaries

Once the binary stream is fully assembled on the worker's disk, the data crosses the execution boundary. The true power of the Rivage engine lies in its ability to run arbitrary user code (Python, Rust, Bash, etc.) without requiring the user to install a heavy middleware SDK (such as PySpark or Hadoop libraries).

The daemon leverages the host operating system's POSIX capabilities. It forks a new, isolated process (`os/exec.CommandContext`) utilizing the binary or script path defined in the `TaskSpec`.

To pass the data to this agnostic script, the daemon employs two universal Unix mechanisms:

1. **Standard Input Piping (stdin):** The daemon opens the assembled local file and pipes its byte stream directly into the subprocess's `stdin`. The user's script (e.g., `sys.stdin` in Python) simply reads line-by-line.
2. **Environment Variable Injection:** The daemon injects execution metadata directly into the subprocess environment space. Variables such as `RIVAGE_WORKER_ID`, `RIVAGE_TASK_ID`, and `RIVAGE_AFFINITY_KEYS` allow the user's script to dynamically alter its behavior based on the cluster state without requiring external API calls.

Upon successful execution, the script emits its deterministic results to standard output (`stdout`). The daemon captures this stream and writes it to an isolated output file on the local disk. If the user's script crashes, it emits a non-zero exit code. The daemon intercepts this exit code, halts the `stdout` capture, and reports a `TASK_STATUS_FAILED` message to the Coordinator, triggering the Scheduler's retry matrix.

3.7.5 The Task Execution Algorithm

The complete lifecycle from chunk ingestion to process execution and upstream transmission is formally defined in Algorithm 4.

Input: A buffered gRPC stream S , System Semaphore Σ

Output: A deterministic binary output stream or execution failure

```

1:  $T_{\text{spec}}, F_{\text{in}} \leftarrow \text{ReassembleFromStream}(S)$ 
2:
4:  $\text{Acquire}(\Sigma)$  // Block until a CPU core is available
5:  $\text{ActiveTasks.Add}(1)$ 
6:
8:  $\text{Ctx} \leftarrow \text{WithTimeout}(\text{Background}(), T_{\text{spec}}.\text{Timeout})$ 
9:  $P \leftarrow \text{exec.CommandContext}(\text{Ctx}, T_{\text{spec}}.\text{Command}, T_{\text{spec}}.\text{Args})$ 
10:
11:  $P.\text{Env} \leftarrow \text{Inject}(T_{\text{spec}}.\text{EnvVars})$ 
12:  $P.\text{Stdin} \leftarrow \text{OpenFile}(F_{\text{in}}, \text{ReadOnly})$ 
13:  $F_{\text{out}} \leftarrow \text{CreateTempFile}()$ 
14:  $P.\text{Stdout} \leftarrow F_{\text{out}}$ 
15:
17:  $E_{\text{err}} \leftarrow P.\text{Run}()$ 
18:
19:  $\text{Release}(\Sigma)$  // Relinquish the CPU core immediately
20:  $\text{ActiveTasks.Sub}(1)$ 
21:
23: if  $E_{\text{err}} \neq \text{null}$  or  $\text{Ctx.Err}() \neq \text{null}$  then
24:    $\text{SendStatus}(S, \text{FAILED})$ 
25:    $\text{CleanupFiles}(F_{\text{in}}, F_{\text{out}})$ 
26:   return
27:
28:  $\text{StreamFileUpstream}(S, F_{\text{out}}, \text{ChunkSize} = 512\text{KB})$ 
29:  $\text{CleanupFiles}(F_{\text{in}}, F_{\text{out}})$ 

```

Figure 11: Pseudocode representing the bounded-memory POSIX execution pipeline within the Worker Daemon.

3.7.6 Telemetry and The Heartbeat Loop

In parallel to the task execution pipeline, the Worker Daemon runs an isolated, high-priority telemetry loop. For the Coordinator's Scheduler to make mathematically sound load-balancing decisions, it requires physical cluster data that is no more than a few milliseconds old.

Every 500 milliseconds, a background goroutine probes the underlying host hardware. It measures physical system memory availability and calculates the average CPU core saturation over the sliding window. This data, alongside the atomic `ActiveTasks` integer, is packaged into a `WorkerMessage_Heartbeat` payload.

Because the daemon utilizes a multiplexed HTTP/2 connection, it can inject these tiny heartbeat frames directly into the active gRPC stream, interleaving them seamlessly between the 512KB data chunks of a returning result payload. This guarantees that hardware telemetry is strictly continuous and never subjected to TCP Head-of-Line (HoL) blocking, ensuring absolute routing precision at the Control Plane.

3.8 Fault Tolerance and System Resilience

In distributed computing, hardware volatility is a statistical certainty rather than an edge case. Commodity compute nodes experience network partitions, memory faults, and kernel panics. To maintain exactly-once processing semantics and prevent cluster-wide deadlocks, the Rivage engine implements a rigorous, two-tiered fault tolerance model, isolating failures across the Compute Plane and the Control Plane.

3.8.1 Compute Plane Resilience (Worker Failures)

Worker daemons operate in an environment where arbitrary, user-defined polyglot scripts are executed. These scripts may contain infinite loops, memory leaks, or segmentation faults. The orchestration engine must guarantee that a catastrophic failure in a worker process does not corrupt the global DAG state.

3.8.1.1 POSIX Isolation and Timeout Enforcement

As detailed in the Worker Daemon architecture, user scripts are never executed within the daemon's native Go memory space. By leveraging operating system-level process forking (`os/exec`), the engine enforces a strict physical boundary.

If a user's Python script attempts to allocate more RAM than physically available, the Linux Out-Of-Memory (OOM) killer will terminate the child process (`SIGKILL`), but the Go worker daemon will survive. The daemon intercepts the non-zero exit status, cleans up the temporary disk files, and reports a structured error upstream.

Furthermore, to prevent infinite loops (e.g., a `while True` bug in the user's code), the Coordinator embeds a strict `Timeout` constraint inside every `TaskSpec`. The Worker Daemon enforces this using a context cancellation function. If the system clock exceeds the deadline, the daemon forcefully sends a `SIGTERM` to the child process, releasing the CPU core back to the counting semaphore.

3.8.1.2 The Watchdog and Idempotent Retries

If the worker daemon itself crashes (e.g., power loss to the physical machine), the node will drop off the HTTP/2 gRPC stream and cease transmitting telemetry.

The Coordinator's global Watchdog algorithm detects this by evaluating the `last_heartbeat` timestamp. Because MapReduce tasks are mathematically modeled as pure, idempotent functions, the Coordinator does not need to attempt a complex state rollback. It simply discards the dead node's active tasks, downgrades their state from `Running` to `Pending`, and re-queues them for the next available healthy node. This guarantees eventual completion without side-effect corruption.

3.8.2 Control Plane Resilience (Master Failures)

While worker nodes are inherently stateless and expendable, the Coordinator represents a single point of failure. It holds the topological state of the entire DAG in its volatile RAM. If the master node experiences a power loss, the entire execution graph is destroyed.

Standard distributed systems often rely on external, highly available databases (like Apache Zookeeper or etcd) to store state. However, introducing an external database

violates Rivage’s zero-dependency design philosophy and introduces massive network transaction latency during the high-frequency dispatch loop.

3.8.2.1 Write-Ahead Logging (WAL) Architecture

To achieve durability with $O(1)$ write latency, the Coordinator implements a localized Write-Ahead Log (WAL). Before the master node alters the state machine of any task, it appends a deterministic binary event to a `.wal` file on its local disk.

The WAL operates sequentially and uses an append-only architecture. Because modern NVMe drives handle sequential appends significantly faster than random B-Tree database inserts, the logging overhead is virtually negligible.

```
struct wal_entry {
    timestamp : Integer (Unix Epoch)
    event_type : Byte (Dispatched=1, Completed=2, Failed=3)
    task_id    : String
    worker_id  : String
}

interface JobWAL {
    Log(event: Byte, task: String, worker: String) → Error
    Recover() → []wal_entry
}
```

Every stage in the DAG is assigned its own isolated WAL file. When the `watchJob` algorithm routes a task, it calls `WAL.Log(Dispatched)` **before** sending the payload over the network. When the result is successfully saved to the `ResultStore`, it calls `WAL.Log(Completed)`.

3.8.2.2 The WAL Recovery Algorithm

If the Coordinator node crashes midway through a 10,000-task Map stage, it must reconstruct the exact execution state upon reboot to avoid repeating successfully completed work.

During the initialization of a DAG stage, the Coordinator performs a localized recovery pass. It opens the .wal file and sequentially replays the binary events to rebuild the state machine in memory.

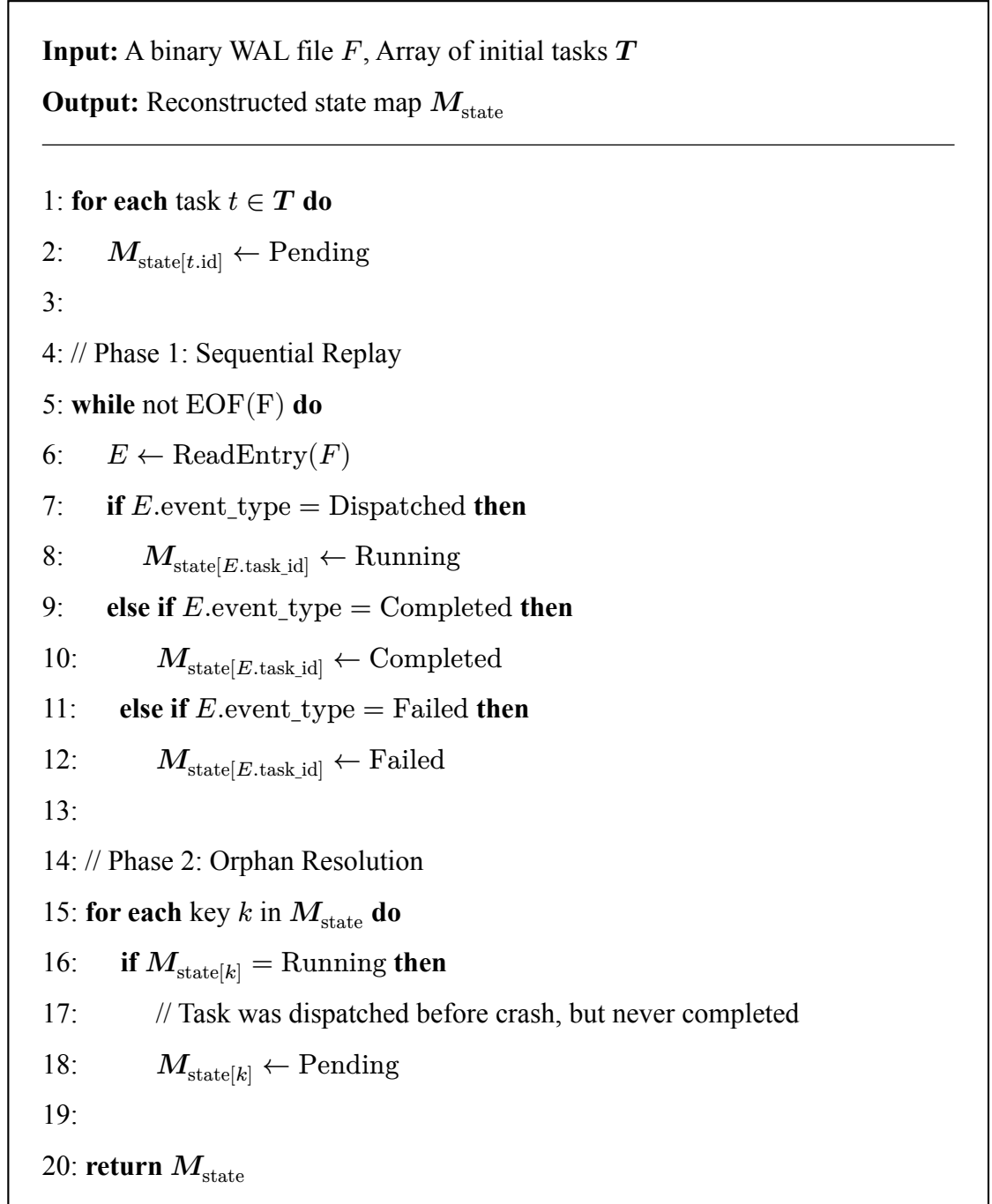


Figure 12: Pseudocode representing the deterministic WAL recovery algorithm executed during master node reboot.

The critical logic occurs in Phase 2 (Orphan Resolution). If the WAL replay finishes and a task is left in the `Running` state, it means the master dispatched the task but crashed before it could record the completion. The master assumes the task was lost or the network connection was severed during the crash. It conservatively downgrades the task back to `Pending`.

By combining idempotency at the Compute Plane with deterministic WAL replays at the Control Plane, the Rivage engine establishes an impenetrable guarantee: regardless of physical hardware failures, network partitions, or kernel panics, the mathematical integrity of the DAG execution will eventually resolve to a correct and consistent output.

3.9 The Programming Model and Programmable Shuffles

While the Coordinator, Scheduler, and Worker Daemon handle the physical execution of the distributed system, the end-user interacts exclusively with the logical Programming Model. Rivage abstracts the complexities of distributed consensus, leaving the developer to define only three components: the execution logic (Polyglot Scripts), the execution topology (the DAG Builder), and the network routing boundaries (Programmable Shuffles).

3.9.1 Polyglot Task Execution

To define a custom Map or Reduce function, a developer simply writes a standalone script in any language. The only contract enforced by the Rivage engine is that the script must read its input payload from standard input (`stdin`) and emit its deterministic result to standard output (`stdout`).

```
import sys, json

def main():
    // 1. Read grouped intermediate data from the orchestrator
    data = json.load(sys.stdin)

    // 2. Perform custom aggregation logic
    target_key = data["key"]
    total_sum = sum(int(val) for val in data["values"])

    // 3. Emit the final result back to the orchestrator
    print(json.dumps({target_key: total_sum}))

if __name__ == "__main__":
    main()
```

Because this script requires zero Rivage-specific SDKs or libraries, it can natively import advanced computational frameworks like `numpy`, `tensorflow`, or custom C++ math bindings without conflicting with the Go orchestrator.

3.9.2 The DAG Builder Interface

To construct a workflow, the developer uses the `dag.Builder` interface within the Go Control Plane. This fluent API allows operators to wire sequential or branching execution stages together.

```
pipeline, err := dag.New("distributed_analytics").
    Stage("map_phase",
        dag.ScriptExecutor("python3", "map.py"),
        dag.WithParallelism(100), // Max concurrent tasks
    ).
    Stage("reduce_phase",
        dag.ScriptExecutor("python3", "reduce.py"),
        dag.WithShuffle(dag.JSONKeyGroupShuffle()),
    ).
    Build()
```

When `pipeline.Build()` is invoked, the engine statically analyzes the declared stages, performs Kahn’s Algorithm to generate the topological execution order, and mathematically guarantees the absence of circular dependencies before any network activity begins.

3.9.3 Programmable Data Routing (ShuffleFunc)

The most computationally expensive phase of any distributed MapReduce pipeline is the network Shuffle. Traditional frameworks strictly enforce a “Sort and Group-By-Key” shuffle paradigm. While effective for counting words or processing text logs, this rigid paradigm is disastrous for dense numerical workloads like distributed matrix multiplication.

To solve this, Rivage exposes a completely programmable data routing boundary via the `ShuffleFunc` interface:

```
type ShuffleFunc func(outputs []TaskOutput) (ShuffleResult, error)
```

By supplying a custom `ShuffleFunc` to a stage, the developer overrides the default MapReduce routing. For example, instead of sorting strings, a developer performing a distributed matrix multiplication can write a custom mathematical router:

```
func MatMulRowBandShuffle(outputs []dag.TaskOutput) (dag.ShuffleResult,
error) {
    // 1. Intercept all completed tile multiplications
    rowGroups := make(map[int][]json.RawMessage)

    for _, out := range outputs {
        meta := parseTileMetadata(out.Data)
        // 2. Group matrix tiles mathematically by their geometric Row
        rowGroups[meta.TileRow] = append(rowGroups[meta.TileRow],
out.Data)
    }

    res := make(dag.ShuffleResult)
    for row, tiles := range rowGroups {
        // 3. Generate a custom Reduce task specifically for this row
band
        taskID := fmt.Sprintf("assemble-row-%d", row)
        res[taskID] = dag.TaskInput{Data: serialize(row, tiles)}
    }
    return res, nil
}
```

This architecture allows Rivage to dynamically morph from a standard MapReduce text processor into a highly specialized, grid-based High-Performance Computing (HPC) engine simply by altering the `ShuffleFunc` at the DAG initialization phase, eliminating the need for specialized external frameworks.

CHAPTER 4

RESULTS AND DISCUSSION

[To be written: Performance benchmarks, evaluation of MapReduce jobs (e.g., word count, batch processing) running on distributed nodes. Analysis of system throughput and resource utilization.]

CHAPTER 5

CONCLUSION AND FUTURE WORKS

[To be written: Summary of findings regarding the efficiency of the polyglot approach. Future scope will include implementing fault tolerance, advanced load-aware scheduling, and external disk-based shuffling.]

CHAPTER 5

REFERENCES

- [1] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [2] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Spark: Cluster computing with working sets,” in *HotCloud*, 2010, p. 95.
- [3] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, “ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging,” *ACM Transactions on Database Systems (TODS)*, vol. 17, no. 1, pp. 94–162, 1992.
- [4] A. D. Birrell and B. J. Nelson, “Implementing Remote Procedure Calls,” *ACM Transactions on Computer Systems (TOCS)*, vol. 2, no. 1, pp. 39–59, 1984.
- [5] gRPC Authors, “gRPC: A high performance, open source universal RPC framework.” [Online]. Available: <https://grpc.io/>
- [6] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18-20, 1967, spring joint computer conference*, 1967, pp. 483–485.

CHAPTER 1

APPENDIX A: Output

[To be written: Tables, extended code snippets (e.g., Protobuf definitions, worker core logic), and supplementary data not included in the main report.]